

Java: Encapsulation `static` Keyword

Mladen Ivkovic

COMP2221 Programming Paradigms –
Object-oriented Programming

Lecture 2 Recap

Join by
Web

PollEv.com
/mladenivkovic356

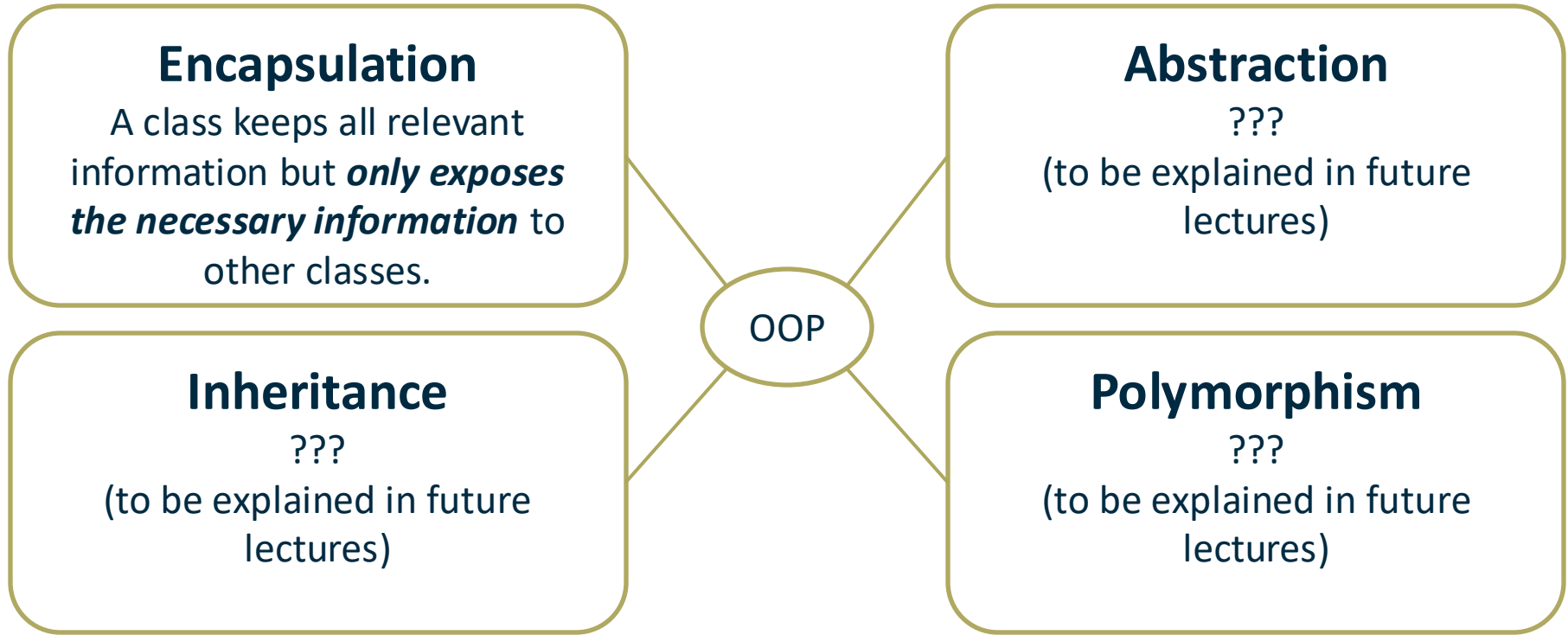


What Will You Learn in this Lecture?

- What are the four key principles of OOP?
- What is encapsulation?
- How do we provide it in Java?
- The use of the *static* keyword

OOP and Encapsulation

The Four Principles of OOP



Encapsulation

- Encapsulation is a **fundamental concept** in **Object-Oriented Programming (OOP)**
- restricts **direct access** to some object components.
- This helps maintain **data integrity** and **security**.
- Java achieves this through **access modifiers**, primarily *private* and *public*.

Encapsulation

Definition:

Encapsulation is the ***bundling of data (fields) and methods*** that operate on the data into a ***single unit (class)*** while ***restricting direct access*** to some of the object's components.

- **Why Encapsulation?**

- Protects data from **accidental/erroneous modification**.
- Enhances **data hiding** and **security**.
- Promotes **modular** and **maintainable** code.

Access Modifiers

Access Modifiers (*public* and *private*)

- Enabling the access control of variable and method from other classes
 - Only allowing other classes to access the least amount of information for getting their jobs done
- Essentially, they are:
 - *public*
 - *private*
 - (and *protected*, but you do not need this for now)

Access Modifiers in Java

- (Focus on `private` & `public` for now – there are more.)

Modifier	Scope
<code>private</code>	Accessible only within the same class
<code>public</code>	Accessible from anywhere in the program

Example: Without Encapsulation

```
class Person {
    String name;    // No restriction
    int age;        // No restriction

    void display() {
        System.out.println("Name: " + name +
                           ", Age: " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person();
        p.name = "Alice";    // Direct access:
                             // it could have been "Malice"
        p.age = -5;          // Invalid age (no validation)
        p.display();        // Incorrect output possible
    }
}
```



Do NOT do this! This is bad practice!

Problems:

- The age field can be set to **invalid values**.
- There is **no control** over what is assigned to name and age.
- No **data hiding** - fields are **directly accessible**.

Encapsulation with *private* Fields and *public* Methods

- The **correct way** to use encapsulation (next slide)

```

class Person {
    private String name; // Restricted access
    private int age;      // Restricted access

    // Setter for name
    public void setName(String newName) {
        name = newName;
    }

    // Getter for name
    public String getName() {
        return name;
    }

    // Setter for age (with validation)
    public void setAge(int newAge) {
        if (newAge > 0) {
            age = newAge;
        } else {
            System.out.println("Invalid age!");
        }
    }

    // Getter for age
    public int getAge() {
        return age;
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Person p = new Person();
        p.setName("Alice"); // Setting the name
        p.setAge(-5);        // Invalid age check
        p.setAge(25);        // Corrected age

        System.out.println("Name: " + p.getName());
        System.out.println("Age: " + p.getAge());
    }
}

```



Benefits of Encapsulation:

- **Data hiding:** Direct access to fields is prevented.
- **Validation:** The `setAge()` method ensures only valid values are stored.
- **Controlled access:** The class controls how its data is modified.

Using Constructors for Encapsulation

- Encapsulation can also be achieved using **constructors** instead of getter/setter methods.
 - Data remains hidden
 - Access remains controlled
 - Adds validation

See the next slide.

Using Constructors for Encapsulation

```
class Car {  
    private String brand;  
    private int speed;  
  
    // Constructor with validation  
    public Car(String brand, int speed) {  
        this.brand = brand;  
        if (speed >= 0) {  
            this.speed = speed;  
        } else {  
            this.speed = 0; // Default to zero if negative  
        }  
    }  
  
    public String getBrand() { return brand; }  
  
    public int getSpeed() { return speed; }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
  
        // Negative speed corrected  
        Car myCar = new Car("Toyota", -10);  
  
        System.out.println("Brand: " +  
                             myCar.getBrand());  
  
        System.out.println("Speed: " +  
                             myCar.getSpeed());  
    }  
}
```


`this` keyword

- Used to refer to the **current object** in a method or constructor.
- Common usage: eliminate confusion between class attributes and parameters with the same name

```
class Car {  
    private String brand;  
    private int speed;  
  
    // Constructor with validation  
    public Car(String brand) {  
        this.brand = brand;  
    }  
}
```

Summary of Key Points

Feature	Without Encapsulation	With Encapsulation
Data Hiding	✗ No protection	✓ Secure and private
Validation	✗ Can set invalid values	✓ Controlled via methods
Code Maintainability	✗ Harder to maintain	✓ Easier to modify and expand

static Keyword

Understanding the `static` Keyword

Objective:

By the end of this lecture, you will understand:

- ✓ What `static` means in Java
- ✓ How `static` affects **variables, methods, and blocks**
- ✓ The difference between **instance members** and **static members**
- ✓ Practical use cases of `static` in **real-world applications**

What is static in Java?

Definition:

The static keyword in Java is used to create **class-level members** that **belong to the class** rather than an *instance* of the class.

Key Concept:

- **Instance members** → belong to objects (each object gets its own copy).
- **Static members** → belong to the **class itself** (shared across all instances).

Example in the next slide

static in Java: example

```
class Example1 {  
    static int staticVar = 10; // Shared across all objects  
    int instanceVar = 20;      // Separate for each object  
}
```

```
class Calculator {  
    static float pi = 3.14159; // Shared across all objects  
    Float instanceVar = 0.;     // Separate for each object  
}
```

static Variables (Class Variables)

- A **static variable** is shared among **all instances** of a class.
- **Only one copy** of the static variable exists, regardless of how many objects are created.

static Variables (Class Variables): example

```
class Counter {  
    static int count = 0; // Static variable (shared)  
  
    Counter() {  
        count++; // Increments shared variable  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Counter obj1 = new Counter();  
        Counter obj2 = new Counter();  
        Counter obj3 = new Counter();  
        System.out.println("Total objects created: " + Counter.count);  
    }  
}
```

What is the output?

static Variables (Class Variables): example

```
class Counter {  
    static int count = 0; // Static variable (shared)  
  
    Counter() {  
        count++; // Increments shared variable  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Counter obj1 = new Counter();  
        Counter obj2 = new Counter();  
        Counter obj3 = new Counter();  
        System.out.println("Total objects created: " + Counter.count);  
    }  
}
```

Output:
Total objects created: 3

static keyword: Key Takeaways

- Each object **does not** get its own copy of the count; it's **shared**.
- static variables are **memory-efficient** because they don't get re-created for each object.

`static final` - Constants

- The `static final` combination is used for **constants** (values that never change).
- **Classic Example: Defining Constants**

static final Constants

Classic Example: Defining Constants

```
class Config {  
    static final double PI = 3.14159; // Constant  
}
```

Key Takeaways:

- **Cannot be modified after initialization.**
- Used for **global constants** (e.g., `Math.PI`).

static Methods:

Example: Utility Methods (Math Library)

- A **static method** belongs to the class, **not to objects**.
- It **cannot access instance (non-static) variables/methods** directly.
- **Why?**

static Methods:

Example: Utility Methods (Math Library)

- A **static method** belongs to the class, **not to objects**.
- It **cannot access instance (non-static) variables/methods** directly.
- **Why?** Because static methods exist **before** objects are created.

static Methods:

```
class MathUtil {  
    static int square(int x) {  
        // only using parameter x, no class variables!  
        return x * x;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        // Note the use without instantiating an object!  
        System.out.println(MathUtil.square(5)); // Output: 25  
    }  
}
```

static Methods: Incorrect Example

(✗ Do NOT do this at home!)

```
class Example {  
    int num = 10;  
  
    static void display() {  
        System.out.println(num); // ERROR: Cannot access  
                                   // non-static variable  
    }  
}
```


static Methods: Incorrect Example

How do we fix this?

```
class Example {  
    int num = 10;  
  
    static void display() {  
        System.out.println(num); // ERROR: Cannot access  
                                   // non-static variable  
    }  
}
```

static Methods: Incorrect Example

Fix: Convert *num* to *static* or remove *static* from *display()*.

```
class Example {  
    int num = 10;  
  
    static void display() {  
        System.out.println(num); // ERROR: Cannot access  
                                   // non-static variable  
    }  
}
```

static Methods: Key Takeaways

- `static` methods are **commonly used for utility/helper functions** (e.g., `Math.sqrt()`).
- They **cannot use *this* or *super*** *(since they belong to the class, not objects).
- *the **super** construct will be practice soon!

Summary of static Keyword

Feature	Behaviour
static Variable	Shared across all instances (only one copy exists)
static Method	Belongs to the class, cannot access instance members
static final	Defines constants

Summary

- Encapsulation in Java
- Static keyword



The End

Any Questions?



Exercises

- Either tell me what the output of the code is
- Or tell me why the code is wrong and how to fix it

What is the output of this code?

```
class Car {  
    private String brand;  
  
    public Car(String brand) {  
        this.brand = brand;  
    }  
  
    public void printInfo() {  
        System.out.println("brand: " + brand);  
    }  
}
```

```
class Main {  
    public static void main(String[] args){  
        Car myCar = new Car("Mercedes");  
        myCar.printInfo();  
    }  
}
```


What is the output of this code?

```
class Car {  
    private String brand;  
  
    public Car(String brand) {  
        this.brand = brand;  
    }  
  
    public void printInfo() {  
        System.out.println("brand: " + brand);  
    }  
}
```

```
class Main {  
    public static void main(String[] args){  
        Car myCar = new Car("Mercedes");  
        myCar.printInfo();  
    }  
}
```

Output:
brand: Mercedes

What is the output of this code?

```
class MyMath {  
    // round Pi down to 3 so we can  
    // do the maths in our head :)  
    static float pi = 3.f;  
  
    static float circle_area(float radius){  
        return pi * radius * radius;  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        float area = circle_area(2.f);  
        System.out.println(area);  
    }  
}
```

What is the output of this code?

```
class MyMath {  
    // round Pi down to 3 so we can  
    // do the maths in our head :)  
    static float pi = 3.f;  
  
    static float circle_area(float radius){  
        return pi * radius * radius;  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        float area = circle_area(2.f);  
        System.out.println(area);  
    }  
}
```

Output:

The method `circle_area(float)` is undefined for the type `MyMain`

What is the output of this code?

```
class Car {  
    private String brand;  
  
    public Car(String brand) {  
        this.brand = brand;  
    }  
  
    public void printInfo() {  
        System.out.println("Brand:" + brand);  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
        Car newCar = new Car("Porsche");  
        newCar.brand = "Ferrari";  
        newCar.printInfo();  
    }  
}
```

What is the output of this code?

```
class Car {  
    private String brand;  
  
    public Car(String brand) {  
        this.brand = brand;  
    }  
  
    public void printInfo() {  
        System.out.println("Brand:" + brand);  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
        Car newCar = new Car("Porsche");  
        newCar.brand = "Ferrari";  
        newCar.printInfo();  
    }  
}
```

Output:
The field Car.brand is not visible

What is the output of this code?

```
class Car {  
    private String brand;  
  
    public Car(String brand) {  
        brand = brand;  
    }  
  
    public void printInfo() {  
        System.out.println("brand: " + brand);  
    }  
}
```

```
class Main {  
    public static void main(String[] args){  
  
        Car myCar = new Car("Mercedes");  
        myCar.printInfo();  
  
    }  
}
```

What is the output of this code?

```
class Car {  
    private String brand;  
  
    public Car(String brand) {  
        brand = brand;  
    }  
  
    public void printInfo() {  
        System.out.println("brand: " + brand);  
    }  
}
```

```
class Main {  
    public static void main(String[] args){  
  
        Car myCar = new Car("Mercedes");  
        myCar.printInfo();  
  
    }  
}
```

Output:
brand: null

What is the output of this code?

```
class MyMath {  
    // round Pi down to 3 so we can  
    // do the maths in our head :)  
    float pi = 3.f;  
  
    static float circle_area(float radius){  
        return pi * radius * radius;  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        float area = MyMath.circle_area(2.f);  
        System.out.println(area);  
    }  
}
```


What is the output of this code?

```
class MyMath {  
    // round Pi down to 3 so we can  
    // do the maths in our head :)  
    float pi = 3.f;  
  
    static float circle_area(float radius){  
        return pi * radius * radius;  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        float area = MyMath.circle_area(2.f);  
        System.out.println(area);  
    }  
}
```

Output:

Cannot make a static reference to the non-static field pi

What is the output of this code?

```
class Car {  
    static String brand;  
  
    public Car(String brand_) {  
        brand = brand_;  
    }  
  
    public void printInfo() {  
        System.out.println("Brand: "+ brand);  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
        Car newCar1 = new Car("Porsche");  
        Car newCar2 = new Car("Ferrari");  
  
        newCar1.printInfo();  
        newCar2.printInfo();  
    }  
}
```

What is the output of this code?

```
class Car {  
    static String brand;  
  
    public Car(String brand_) {  
        brand = brand_;  
    }  
  
    public void printInfo() {  
        System.out.println("Brand: "+ brand);  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
        Car newCar1 = new Car("Porsche");  
        Car newCar2 = new Car("Ferrari");  
  
        newCar1.printInfo();  
        newCar2.printInfo();  
    }  
}
```

Output:
Brand: Ferrari
Brand: Ferrari

What is the output of this code?

```
class MyMath {  
    // round Pi down to 3 so we can  
    // do the maths in our head :)  
    final float pi = 3.f;  
  
    static float circle_area(float radius){  
        return pi * radius * radius;  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        float area = MyMath.circle_area(2.f);  
        System.out.println(area);  
    }  
}
```

What is the output of this code?

```
class MyMath {  
    // round Pi down to 3 so we can  
    // do the maths in our head :)  
    final float pi = 3.f;  
  
    static float circle_area(float radius){  
        return pi * radius * radius;  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        float area = MyMath.circle_area(2.f);  
        System.out.println(area);  
    }  
}
```

Output:

Cannot make a static reference to the non-static field pi

What is the output of this code?

```
class Car {  
    private static String brand;  
  
    public Car(String brand_) {  
        brand = brand_;  
    }  
  
    public void printInfo() {  
        System.out.println("Brand: "+ brand);  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        Car newCar1 = new Car("Porsche");  
        newCar1.brand = "Ferrari";  
  
        newCar1.printInfo();  
    }  
}
```

What is the output of this code?

```
class Car {  
    private static String brand;  
  
    public Car(String brand_) {  
        brand = brand_;  
    }  
  
    public void printInfo() {  
        System.out.println("Brand: "+ brand);  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        Car newCar1 = new Car("Porsche");  
        newCar1.brand = "Ferrari";  
  
        newCar1.printInfo();  
    }  
}
```

Output:

The field `Car.brand` is not visible

What is the output of this code?

```
class MyMath {  
    // round Pi down to 3 so we can  
    // do the maths in our head :)  
    final float pi = 3.f;  
  
    float circle_area(float radius){  
        return pi * radius * radius;  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        float area = MyMath.circle_area(2.f);  
        System.out.println(area);  
    }  
}
```


What is the output of this code?

```
class MyMath {  
    // round Pi down to 3 so we can  
    // do the maths in our head :)  
    final float pi = 3.f;  
  
    float circle_area(float radius){  
        return pi * radius * radius;  
    }  
}
```

```
public class MyMain {  
    public static void main(String[] args) {  
  
        float area = MyMath.circle_area(2.f);  
        System.out.println(area);  
    }  
}
```

Output:

Cannot make a static reference to the non-static method circle_area(float) from the type MyMath